Graf algoritmi

Zadaci za vježbanje: rekurzija

Težina zadatka označena je zvjezdicama uz broj zadatka: * lagano, ** srednje, *** teže. Ako nije drukčije navedeno, funkcije se pišu rekurzivno, bez petlji. **Većina laganih zadataka (*) tipično se rješava iterativno, ovdje su samo zbog vježbanja pisanja rekurzivnih funkcija. Takve zadatke možete za provjeru riješiti i iterativno.**

Brojevi

1.* Napisati funkciju koja za prirodni broj `n` ispisuje brojeve od `n` do 1, a zatim ponovno do `n`.

Primjer: za `n = 3` ispis je `3 2 1 2 3`.

2.* Napisati funkciju koja vraća listu svih neparnih brojeva od 1 do `n`.

Primjer: `odd_up_to(7)` vraća `[1, 3, 5, 7]`.

3.* Napisati funkciju koja vraća broj neparnih znamenki prirodnog broja.

Primjer: za 248753 rezultat je 3.

4.* **Repunit** je broj čije su sve znamenke 1. Napisati funkciju koja provjerava je li prirodni broj repunit.

Primjer: za 1111 rezultat je `True`, a za 121 `False`.

5.* Najveći zajednički djelitelj računa se Euklidovim algoritmom: $\gcd(a, 0) = a$, a $\gcd(a, b) = \gcd(b, a \bmod b)$. Napisati funkciju `gcd(a, b)`.

Primjer: `gcd(1071, 462)` vraća 21.

6.** Broj je **koračni** ako se svake dvije susjedne znamenke razlikuju za 1. Napisati funkciju koja provjerava je li broj koračni.

Primjer: za 4321 i 45654 rezultat je `True`, a za 421 `False`.

7.** Napisati funkciju koja vraća u koliko se koraka množenjem znamenki od broja `n` dolazi do jednoznamenkastog broja.

Primjer: za 39 rezultat je 3 (39, 27, 14, 4), a za 999 rezultat je 4.

Riješeni primjer: Fibonaccijevi brojevi. Vrijedi $F_0 = 0$, $F_1 = 1$ i $F_n = F_{n-1} + F_{n-2}$. Obična rekurzija:

```
def fib(n):
    if n < 2:
        return n
    return fib(n - 1) + fib(n - 2)
```

S brojem poziva: funkcija vraća tuple (rezultat, broj poziva). Svaki poziv broji sebe i pozive iz oba podpoziva.

```
def fib_calls(n):
    if n < 2:
        return n, 1
    a, calls_a = fib_calls(n - 1)
    b, calls_b = fib_calls(n - 2)
    return a + b, calls_a + calls_b + 1
```

S memoizacijom: izračunate vrijednosti spremaju se u dictionary, pa se svaka računa samo jednom.

```
def fib_memo(n, memo=None):
    if memo is None:
        memo = {}
    if n < 2:
        return n, 1
    if n in memo:
        return memo[n], 1
    a, calls_a = fib_memo(n - 1, memo)
    b, calls_b = fib_memo(n - 2, memo)
    memo[n] = a + b
    return memo[n], calls_a + calls_b + 1
```

Za `n = 30` obje funkcije vraćaju 832040, ali `fib_calls` uz 2692537 poziva, a `fib_memo` uz samo 59. Parametar `memo` ima zadanu vrijednost `None`, a ne `{}`, jer bi se prazan dictionary kao zadana vrijednost dijelio između svih poziva funkcije.

8.** Binomni koeficijent računa se po Pascalovom trokutu: $\binom{n}{0} = \binom{n}{n} = 1$, a $\binom{n}{k} = \binom{n-1}{k-1} + \binom{n-1}{k}$.

(a) Napisati funkciju `binom(n, k)` koja vraća tuple: binomni koeficijent i broj poziva funkcije.

(b) Napisati verziju s memoizacijom (izračunate vrijednosti spremaju se u dictionary) koja također vraća tuple, pa usporediti broj poziva.

Primjer: `binom(10, 5)` vraća `(252, 503)`, a `binom(20, 10)` treba 369511 poziva.

9.** **Penjanje stepenicama.** Stepenice se penju koracima od 1 ili 2 stepenice.

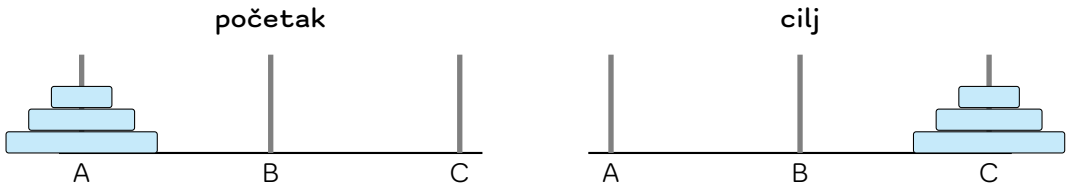
(a) Napisati funkciju koja vraća tuple: broj načina na koji se može popeti `n` stepenica i broj poziva funkcije.

(b) Napisati verziju s memoizacijom koja također vraća tuple, pa usporediti broj poziva.

(c) Riješiti isti zadatak za korake od 1, 2 ili 3 stepenice.

Primjer: za `n = 10` rezultat je `(89, 177)` bez memoizacije i `(89, 19)` s memoizacijom, za `n = 30` bez memoizacije potrebno je 2692537 poziva, a s koracima 1, 2 ili 3 za `n = 10` ima 274 načina.

10.** **Hanojski tornjevi.** Na štapu `A` je `n` diskova, od najvećeg na dnu do najmanjeg na vrhu. Treba ih premjestiti na štap `C`, uz pomoćni štap `B`. U jednom potezu premješta se jedan disk, a veći disk nikad ne smije biti na manjem. Napisati funkciju koja vraća listu poteza.



Primjer: za `n = 3` potezi su `A-C, A-B, C-B, A-C, B-A, B-C, A-C`, a za `n = 10` ima ih `1023`.

Stringovi i liste

11.* Napisati funkciju koja vraća obrnuti string.

Primjer: `reverse('python')` vraća `'nohtyp'`.

12.* Napisati funkciju koja provjerava je li string palindrom (čita se isto s obje strane).

Primjer: `is_palindrome('anavolimilovana')` vraća `True`, a `is_palindrome('python')` vraća `False`.

13.* Napisati funkciju koja vraća broj samoglasnika u stringu.

Primjer: `vowels('rekurzija')` vraća 4.

14.* Napisati funkciju koja vraća duljinu liste, bez korištenja funkcije `len`.

Primjer: `length([4, 8, 15, 16, 23, 42])` vraća 6.

15.* Napisati funkciju koja vraća zbroj elemenata liste.

Primjer: `list_sum([3, 1, 4, 1, 5])` vraća 14.

16.* Napisati funkciju koja vraća najveći element liste.

Primjer: `maximum([5, 17, 3, 11])` vraća 17.

17.* Napisati funkciju koja vraća listu kvadrata elemenata liste.

Primjer: `squares([1, 2, 3, 4])` vraća `[1, 4, 9, 16]`.

18.* Napisati funkciju koja vraća listu parnih elemenata liste, u istom redoslijedu.

Primjer: `evens([5, 2, 8, 3, 6])` vraća `[2, 8, 6]`.

19.* Napisati funkciju koja provjerava je li lista sortirana uzlazno.

Primjer: `is_sorted([1, 2, 2, 5])` vraća `True`, a `is_sorted([1, 3, 2])` vraća `False`.

20.* Napisati funkciju koja naizmjenično uzima elemente dviju lista. Kad se jedna lista potroši, dodaju se preostali elementi druge.

Primjer: `interleave([1, 2, 3], ['a', 'b', 'c'])` vraća `[1, 'a', 2, 'b', 3, 'c']`, a `interleave([1, 2], ['a', 'b', 'c', 'd'])` vraća `[1, 'a', 2, 'b', 'c', 'd']`.

21.** Napisati funkciju koja iz liste uklanja uzastopna ponavljanja istog elementa.

Primjer: `compress([1, 1, 2, 3, 3, 3, 1])` vraća `[1, 2, 3, 1]`.

22.** Napisati funkciju koja od dviju sortiranih lista napravi jednu sortiranu listu.

Primjer: `merge([1, 4, 9], [2, 3, 10, 12])` vraća `[1, 2, 3, 4, 9, 10, 12]`.

23.** Napisati funkciju koja ugniježđenu listu pretvara u listu bez podlista (funkcija `isinstance(x, list)` provjerava je li `x` lista).

Primjer: `flatten([1, [2, [3, 4]], 5])` vraća `[1, 2, 3, 4, 5]`.

24.** Napisati funkciju koja vraća dubinu ugniježđene liste: lista bez podlista ima dubinu 1.

Primjer: `depth([1, [2, [3]]])` vraća 3, a `depth([1, 2])` vraća 1.

25.** Napisati funkciju koja vraća zbroj svih brojeva u ugniježđenoj listi.

Primjer: `nested_sum([1, [2, [3, 4]], 5])` vraća 15.

26.** Napisati funkciju koja vraća listu svih podskupova liste (redoslijed podskupova nije bitan). Uputa: podskupovi liste su podskupovi ostatka liste, sa i bez prvog elementa.

Primjer: `subsets([1, 2, 3])` vraća 8 podskupova: `[]`, `[3]`, `[2]`, `[2, 3]`, `[1]`, `[1, 3]`, `[1, 2]`, `[1, 2, 3]`, a lista od 10 elemenata ima 1024 podskupa.

27.** Napisati funkciju koja vraća listu svih permutacija stringa. Petlja po znakovima je dopuštena.

Primjer: `permutations('abc')` vraća `['abc', 'acb', 'bac', 'bca', 'cab', 'cba']`, a string od 5 znakova ima 120 permutacija.

Dictionary

Dictionary se obrađuje rekurzivno po listi ključeva `list(d)`: prvi ključ obradi se odmah, a ostali rekurzivnim pozivom.

Riješeni primjer: zbroj vrijednosti. Funkcija vraća zbroj svih vrijednosti u dictionaryju. Lista ključeva prosljeđuje se kao parametar; kod prvog poziva ona je `None`, pa se napravi iz dictionaryja.

```
def value_sum(d, keys=None):
    if keys is None:
        keys = list(d)
    if not keys:
        return 0
    return d[keys[0]] + value_sum(d, keys[1:])

print(value_sum({'a': 3, 'b': 5, 'c': 2}))   # 10
```

28.* Napisati funkciju koja od liste ključeva i liste vrijednosti iste duljine napravi dictionary.

Primjer: `make_dict(['a', 'b', 'c'], [1, 2, 3])` vraća `{'a': 1, 'b': 2, 'c': 3}`.

29.* Napisati funkciju koja za listu riječi vraća dictionary u kojem je svakoj riječi pridružena njezina duljina.

Primjer: `lengths(['graf', 'vrh', 'brid'])` vraća `{'graf': 4, 'vrh': 3, 'brid': 4}`.

30.* Napisati funkciju koja vraća ključ s najvećom vrijednošću.

Primjer: `best({'ana': 7, 'bruno': 9, 'cvita': 4})` vraća `'bruno'`.

31.* Napisati funkciju koja vraća broj ključeva čija je vrijednost veća od zadanog broja `k`.

Primjer: za `{'ana': 7, 'bruno': 9, 'cvita': 4, 'duje': 8}` i `k = 7` rezultat je 2.

32.* Napisati funkciju koja vraća obrnuti dictionary: vrijednosti postaju ključevi, a ključevi vrijednosti (sve vrijednosti su različite).

Primjer: za `{'a': 1, 'b': 2, 'c': 3}` rezultat je `{1: 'a', 2: 'b', 3: 'c'}` (redoslijed ključeva nije bitan).

U sljedećim zadacima petlje su dopuštene za prolaz kroz elemente dictionaryja ili liste.

33.** Napisati funkciju koja za string vraća dictionary u kojem je svakom znaku pridružen broj njegovih pojavljivanja.

Primjer: `char_count('banana')` vraća `{'b': 1, 'a': 3, 'n': 2}`.

34.** **Hijerarhija u firmi.** Dictionary svakoj osobi pridružuje listu njezinih izravno podređenih:

```
team = {'ana': ['bruno', 'cvita'], 'bruno': ['duje', 'ema'],
        'cvita': ['frane'], 'duje': [], 'ema': [],
        'frane': ['goran'], 'goran': []}
```

(a) Napisati funkciju koja vraća listu svih podređenih zadane osobe (izravnih i neizravnih).

(b) Napisati funkciju koja vraća broj razina hijerarhije ispod zadane osobe, uključujući nju.

Primjer: `ana` ima 6 podređenih, `cvita` ima podređene `['frane', 'goran']`; ispod `ana` ima 4 razine (`ana`, `cvita`, `frane`, `goran`), a ispod `bruno` 2.

35.** **Mape i datoteke.** Sadržaj mape zapisan je kao ugniježđeni dictionary: ključ je ime datoteke ili mape, a vrijednost veličina datoteke ili dictionary sa sadržajem mape.

```
fs = {'docs': {'a.txt': 3, 'b.txt': 5},
      'img': {'x.png': 10, 'raw': {'y.png': 20}},
      'notes.txt': 2}
```

(a) Napisati funkciju koja vraća ukupnu veličinu svih datoteka u mapi.

(b) Napisati funkciju koja vraća putanju do zadane datoteke kao listu imena ili `None` ako datoteke nema.

(c) Napisati funkciju koja ispisuje sadržaj mape s uvlakama, kao u primjeru.

Primjer: ukupna veličina je 40 (za mapu `img` 30), putanja do `y.png` je `['img', 'raw', 'y.png']`, a ispis je

```
docs/
    a.txt (3)
    b.txt (5)
img/
    x.png (10)
    raw/
        y.png (20)
notes.txt (2)
```

Matrice

Matrica se obrađuje rekurzivno, redak po redak: prvi redak obradi se odmah, a ostatak matrice (`m[1:]`) rekurzivnim pozivom. Za obradu jednog retka mogu se koristiti funkcije iz prethodnih zadataka, ugrađene funkcije (`sum`, `max`, `count`) i list comprehension.

```python
def matrix_sum(m):
    if not m:               # matrica bez redaka
        return 0
    return sum(m[0]) + matrix_sum(m[1:])

print(matrix_sum([[1, 2, 3], [4, 5, 6]]))   # 21
```

36.* Napisati funkciju koja ispisuje matricu, svaki redak u svom retku.

Primjer: za `[[1, 2, 3], [4, 5, 6]]` ispis je

```
1 2 3
4 5 6
```

37.* Napisati funkciju koja vraća najveći element matrice.

Primjer: za `[[3, 8, 1], [9, 2, 7]]` rezultat je 9.

38.* Napisati funkciju koja vraća broj nula u matrici.

Primjer: za `[[0, 1, 0], [2, 0, 3]]` rezultat je 3.

39.* Napisati funkciju koja vraća novu matricu u kojoj je svaki element pomnožen brojem `k`.

Primjer: za `[[1, 2], [3, 4]]` i `k = 3` rezultat je `[[3, 6], [9, 12]]`.

40.* Napisati funkciju koja vraća listu zbrojeva redaka matrice.

Primjer: za `[[1, 2, 3], [4, 5, 6]]` rezultat je `[6, 15]`.

41.* Napisati funkciju koja vraća broj parnih elemenata matrice.

Primjer: za `[[1, 2, 3], [4, 5, 6]]` rezultat je 3.

42.* Napisati funkciju koja provjerava sadrži li matrica broj `x`.

Primjer: za `[[1, 2], [3, 4]]` i `x = 3` rezultat je `True`, a za `x = 5` `False`.

43.* Napisati funkciju koja vraća stupac `j` matrice kao listu.

Primjer: za `[[1, 2, 3], [4, 5, 6]]` i `j = 1` rezultat je `[2, 5]`.

44.* Napisati funkciju koja vraća novu matricu u kojoj su negativni elementi zamijenjeni nulom.

Primjer: za `[[1, -2], [-3, 4]]` rezultat je `[[1, 0], [0, 4]]`.

45.* Napisati funkciju koja vraća listu elemenata glavne dijagonale kvadratne matrice. Uputa: dijagonala je `m[0][0]` i dijagonala matrice bez prvog retka i prvog stupca.

Primjer: za `[[1, 2, 3], [4, 5, 6], [7, 8, 9]]` rezultat je `[1, 5, 9]`.

46.** Napisati funkciju koja vraća transponiranu matricu (retci postaju stupci). Uputa: prvi redak transponirane matrice je prvi stupac zadane matrice.

Primjer: za `[[1, 2, 3], [4, 5, 6]]` rezultat je `[[1, 4], [2, 5], [3, 6]]`.

47.** Napisati funkciju koja vraća broj putova od gornjeg lijevog do donjeg desnog polja matrice $m \times n$, ako se u svakom koraku ide jedno polje desno ili jedno polje dolje.

Primjer: za matricu 3×3 ima 6 putova, a za 3×7 ima ih 28.

48.** Isto kao u prethodnom zadatku, ali matrica sadrži prepreke: `0` je slobodno polje, a `1` zid.

Primjer: za `[[0, 0, 0], [0, 1, 0], [0, 0, 0]]` ima 2 puta, a za `[[0, 0, 0, 0], [0, 1, 0, 0], [0, 0, 0, 1], [1, 0, 0, 0]]` ima ih 3.

49.*** **Bojenje područja** (bucket fill u programima za crtanje). Slika je matrica znakova. Napisati funkciju koja od zadanog polja sva povezana polja s istim znakom (susjedi su gore, dolje, lijevo i desno) mijenja u zadani znak i vraća broj promijenjenih polja.

Primjer: za sliku

```
..#..
.##..
#..#.
...#.
##.#.
```

bojenjem znakom `*` od polja `(0, 0)` mijenjaju se 3 polja, od polja `(2, 1)` 6 polja, a od polja `(0, 3)` 7 polja.

50.*** **Arhipelag.** U matrici `1` je kopno, a `0` more. Otok čine polja kopna povezana preko susjeda gore, dolje, lijevo i desno. Koristeći funkciju iz prethodnog zadatka, napisati funkciju koja vraća broj otoka i listu njihovih veličina. Petlje su dopuštene.

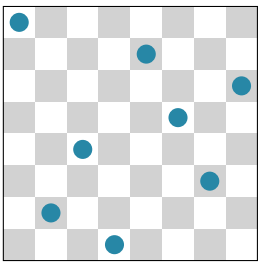
Primjer: za matricu

```
1 1 0 0 0 1
1 0 0 1 0 1
0 0 1 1 0 0
0 0 0 1 0 0
1 0 0 0 0 1
```

ima 5 otoka, veličina `[3, 2, 4, 1, 1]` (otoci redom kako se nailazi na njih po retcima).

Složeniji problemi

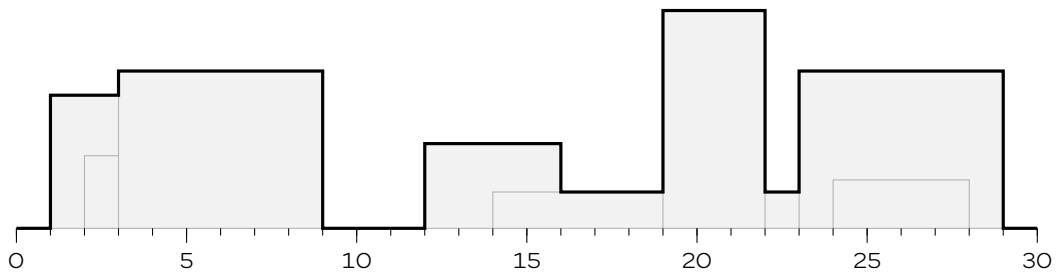
51.*** **Problem n kraljica.** Na šahovsku ploču $n \times n$ treba postaviti `n` kraljica tako da se nikoje dvije ne napadaju (nisu u istom retku, stupcu ni dijagonali). Kraljice se postavljaju redak po redak, a grana se napušta čim nova kraljica napada neku od već postavljenih. Napisati funkciju koja vraća broj rješenja. Petlje su dopuštene.



Na slici je jedno od rješenja za `n = 8`. Zapisano kao lista stupaca po retcima, to je `[0, 4, 7, 5, 2, 6, 1, 3]`.

Primjer: za `n = 1, 2, ..., 8` broj rješenja je `1, 0, 0, 2, 10, 4, 40, 92`.

52.*** **Obris zgrada.** Zadani su položaj i dimenzije `n` pravokutnih zgrada. Treba naći obris zgrada, tj. ukloniti linije koje se ne vide. Dno svih zgrada je na visini `0`. Svaka zgrada je tuple `(l, h, r)`, gdje su `l` i `r` lijeva i desna koordinata, a `h` visina; sve su vrijednosti cijeli brojevi.



(a) Napisati funkciju koja za listu zgrada vraća listu točaka `(x, h)` na kojima se visina obrisa mijenja. Uputa: kao kod merge sorta, lista zgrada dijeli se na dvije polovice, obris svake polovice nalazi se rekurzivno, a zatim se dva obrisa spajaju u jedan. Pri spajanju su petlje dopuštene.

(b) Napisati funkciju koja generira listu zgrada. Parametri su broj zgrada `n`, `max_r` i `max_h`.

(c) Izmjeriti vrijeme izvođenja za različite `n` i procijeniti složenost.

Primjer: za zgrade sa slike

```
[(1, 11, 5), (2, 6, 7), (3, 13, 9), (12, 7, 16),
 (14, 3, 25), (19, 18, 22), (23, 13, 29), (24, 4, 28)]
```

obris je

```
[(1, 11), (3, 13), (9, 0), (12, 7), (16, 3),
 (19, 18), (22, 3), (23, 13), (29, 0)]
```